

NoSQL semestrální projekt
MongoDB sharded cluster nad Yelp
Open Dataset
Lukáš Kašpar

2026

Obsah práce

ÚVOD	3
1. ARCHITEKTURA	4
Obrázkové schema architektury	4
1.1 Schéma a popis architektury	5
1.2 Specifika konfigurace	5
1.2.1 CAP teorém	5
1.2.2 Cluster	6
1.2.3 Uzly	6
1.2.4 Sharding	6
1.2.5 Replikace	7
1.2.6 Perzistence dat	8
1.2.7 Distribuce dat	8
1.2.8 Zabezpečení	10
2. FUNKČNÍ ŘEŠENÍ	11
2.1 Struktura	11
2.1.1 docker-compose.yml	11
2.2 Instalace	12
3. PŘÍPADY UŽITÍ A PŘÍPADOVÉ STUDIE	14
3.1 Pro jaké účely je MongoDB vhodná	14
3.2 Proč byla zvolena MongoDB pro Yelp data	14
3.3 Proč nebyla zvolena jiná NoSQL databáze	14
3.4 Případové studie	14
3.4.1 The Weather Channel	14
3.4.2 Bosch Digital	15
3.4.3 eBay	15
4. VÝHODY A NEVÝHODY	17
4.1 Výhody MongoDB obecně	17
4.2 Nevýhody MongoDB obecně	17
4.3 Výhody tohoto konkrétního řešení	17
4.4 Nevýhody tohoto konkrétního řešení	17
5. DALŠÍ SPECIFIKA	18
6. DATA	19
6.1 Použité datasety	19
6.2 Typy dat a formát	19
6.3 Proč nebyly zvoleny jiné datové struktury	19
6.4 Rozsah dat	19

yelp_business_v1.json.....	20
yelp_review_v1.json	20
yelp_user_v1.json	20
6.5 Prázdné hodnoty	21
6.6 Úpravy dat	21
6.7 Zdroj dat	21
6.8 Python analýza dat	22
7. DOTAZY	26
Práce s daty.....	26
Agregační funkce.....	26
Konfigurační dotazy	26
Práce s poli a vnořenými dokumenty.....	26
Indexy	26
ZÁVĚR	27
ZDROJE.....	27
PŘÍLOHY DOKUMENTACE.....	28

ÚVOD

Tato semestrální práce se zabývá návrhem a implementací distribuovaného NoSQL řešení nad databází MongoDB. Praktická část pracuje se třemi propojitelnými soubory z Yelp Open Dataset: businesses, users a reviews. [1][2][4][5]

Cílem není data pouze uložit, ale navrhnout funkční shardovaný cluster s replikací, validačními schématy, indexací, automatizovaným importem a podporou analytických dotazů nad uloženými dokumenty. [2][4][5][13]

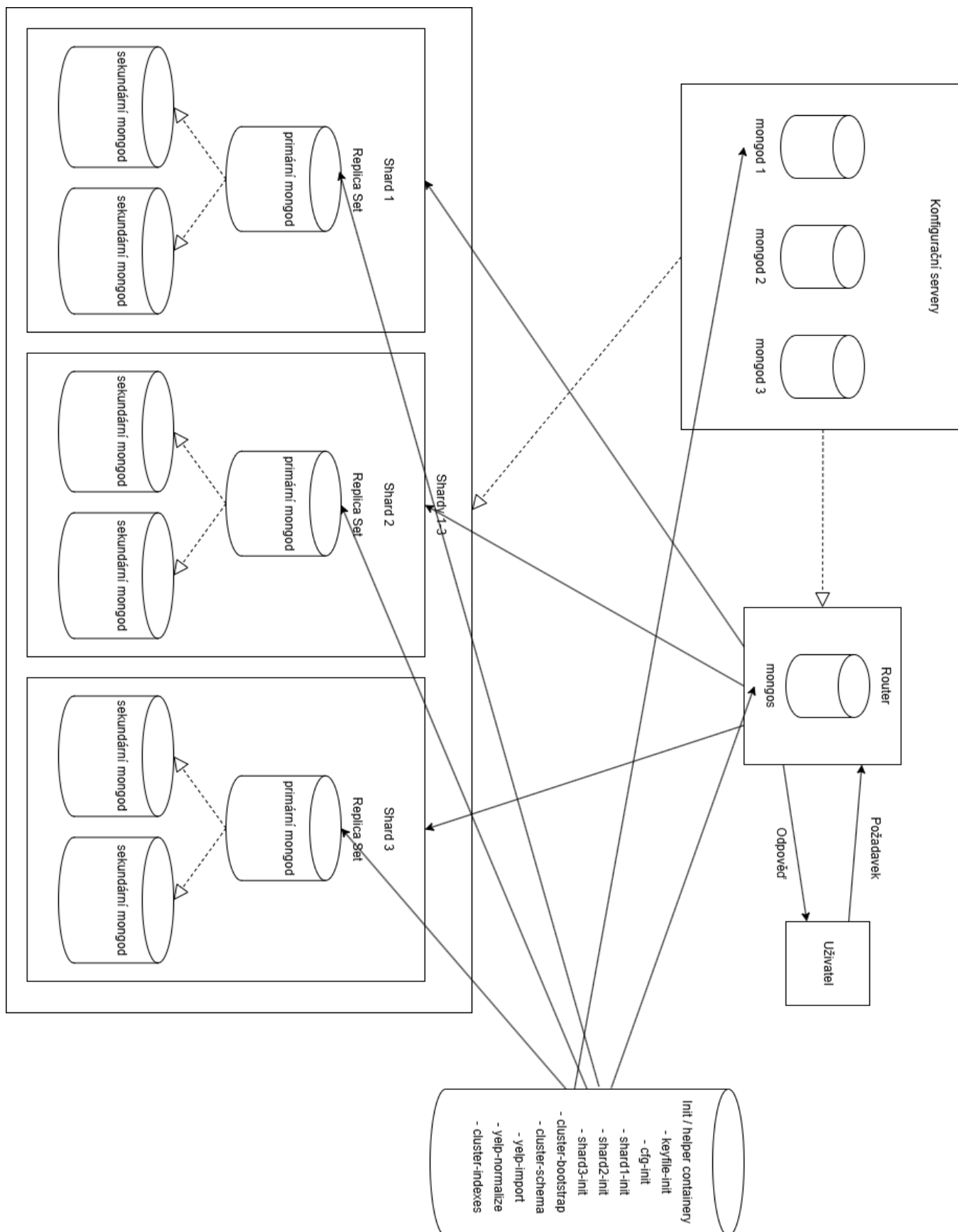
Dokumentace popisuje architekturu řešení v Docker Compose, inicializaci clusteru, strukturu projektu, import a normalizaci dat, zabezpečení, rozdělení dat mezi shardy a sadu netriviálních dotazů. Součástí práce je také základní Python analýza datasetů. [8]

Řešení využívá Docker Compose, image mongo:8.2, pomocný image python:3.12-slim pro generování nefixního keyfile, webové GUI haohanyang/compass-web:0.3.1 a vlastní importní image s MongoDB Database Tools. Volba těchto obrazů vychází z kompatibility, jednoduchosti a snadnější správy. [6][7][8]

Použitá verze MongoDB 8.2 splňuje požadavek zadání. Po vložení datasetů a spuštění docker compose up -d --build proběhne bootstrap clusteru, vytvoření uživatelů, zavedení validačních schémat, shardování kolekcí, import dat, normalizace vybraných polí a vytvoření indexů. Projekt neřeší aplikační vrstvu, TLS, monitoring, zálohování ani benchmarky, protože tyto oblasti přesahují rámec semestrální práce. [6][8][9]

1. ARCHITEKTURA

Obrázkové schema architektury



Obrázek 1 - Architektonické schéma

1.1 Schéma a popis architektury

Architektura řešení je navržena jako jeden shardovaný MongoDB cluster provozovaný v Docker Compose. Cílem je předvést směřování požadavků přes mongos, uchovávání metadat na config serverech, horizontální rozdělení dat mezi shardy a vysokou dostupnost díky replikačním sadám. [2][4][8]

Jádro clusteru tvoří tyto komponenty:

- 1× mongos router – vstupní bod pro klienty a administrační shell,
- 3× config server (cfg1, cfg2, cfg3) – replikační sada uchovávající metadata o clusteru,
- 3× shard – shard01, shard02, shard03, přičemž každý shard je tvořen třemi uzly,
- 9× datový uzel shardů – shard1a–c, shard2a–c, shard3a–c.

Nad touto základní topologií běží sada pomocných služeb, které automatizují bootstrap clusteru, vytvoření uživatelů, zavedení schémat, import, normalizaci dat i tvorbu indexů.

- keyfile-init – vygenerování keyfile pro interní autentizaci mezi uzly,
- cfg-init – inicializace replikační sady config serverů,
- shard1-init, shard2-init, shard3-init – inicializace replikačních sad jednotlivých shardů,
- cluster-bootstrap – vytvoření administrátorského účtu na mongos a připojení shardů do clusteru,
- cluster-schema – vytvoření kolekcí, validačních schémat, shardovacích klíčů a aplikačního uživatele,
- yelp-import – import tří JSON datasetů,
- yelp-normalize – normalizace datových typů a vybraných polí,
- cluster-indexes – vytvoření sekundárních a textových indexů,
- compass-web – webové GUI pro vizuální kontrolu databáze.

Tato architektura je pro semestrální práci vhodná, protože odděluje jednotlivé role databázových služeb a zároveň umožňuje předvést shardování, replikaci, validaci i automatizované spuštění bez ručního zásahu po startu kontejnerů. [2][4][8]

Jak se architektura liší od běžného doporučeného produkčního používání

Řešení je navrženo jako laboratorní a demonstrační varianta, nikoli jako plně produkční enterprise nasazení. Hlavní rozdíly jsou následující: [2][4][8]

1. Pouze jeden veřejně dostupný mongos router. V produkčním prostředí by bylo vhodné provozovat více routerů za load balancerem. V projektu je zvolen jeden router kvůli menší složitosti a nižším nárokům na zdroje.
2. Silná orientace na automatizaci bootstrapu přes jednorázové helper kontejnery. Produkce by typicky používala orchestraci nebo provisioning pipeline, zde je bootstrap součástí Docker Compose workflow.
3. Lokální demonstrační charakter řešení. Chybí TLS, centralizovaný monitoring, alerting, pravidelné zálohy a správa tajemství v produkční kvalitě, protože nejsou nezbytné pro cíle práce.

1.2 Specifika konfigurace

1.2.1 CAP teorém

Z pohledu CAP teorému se řešení blíží variantě CP, tedy preferuje konzistenci a toleranci partitioningu před bezpodmínečnou dostupností. [4][11][17]

MongoDB v tomto návrhu zapisuje přes primární uzel replikační sady. Při výpadku primáru nebo síťovém rozdělení musí nejprve proběhnout volba nového primáru, takže zápisy mohou být dočasně nedostupné. [4][11][17]

Systém tedy nepreferuje absolutní dostupnost za každou cenu, ale korektní stav clusteru a integritu dat.

Config servery i datové shardy běží jako samostatné replikační sady. Dokud replikační sada drží většinu členů, může pokračovat v provozu; bez majority bezpečně nepřijímá zápisy. [4][11][17]

Pro tento projekt je takové chování dostačující, protože důležitější než nepřetržitá dostupnost je správnost importovaných dat, schémat a analytických výstupů.

1.2.2 Cluster

V řešení je použit jeden distribuovaný MongoDB cluster. To je pro účely semestrální práce dostačující, protože umožňuje předvést sharding, replikaci, směrování požadavků i automatizovaný bootstrap v jednom konzistentním prostředí. [2][4]

Použití více samostatných clusterů by zde nepřineslo zásadní funkční výhodu, ale výrazně by zvýšilo složitost konfigurace i dokumentace.

1.2.3 Uzly

Z pohledu databázové vrstvy obsahuje řešení 13 hlavních clusterových služeb:

- 1× mongos,
- 3× config server,
- 9× datový uzel shardů.

Pokud bychom započítali i pomocné kontejnery, Docker Compose definuje více služeb. Pro zadání je však podstatné, že samotná databázová vrstva výrazně překračuje minimum tří uzlů a je rozdělena do více logických vrstev.

Tato konfigurace byla zvolena proto, aby:

- config servery měly kvorum a nebyly single point of failure,
- každý shard měl vlastní replikační sadu,
- cluster mohl současně demonstrovat horizontální rozdělení dat i odolnost proti výpadku jednotlivých uzlů,
- bylo možné provádět administrační a diagnostické dotazy nad stavem shardingu a replikace.

1.2.4 Sharding

Řešení využívá tři shardy. Sharding je zapnut na úrovni aplikační databáze yelp a následně jsou shardovány tři hlavní kolekce:

- businesses pomocí klíče { business_id: "hashed" },
- users pomocí klíče { user_id: "hashed" },
- reviews pomocí klíče { business_id: "hashed" }.

Volba hashed shard key je v projektu vhodná proto, že identifikátory business_id a user_id mají vysokou kardinalitu a vedou k rovnoměrnějšímu rozložení dokumentů mezi shardy. U kolekce reviews je zvolen shard key business_id, protože recenze přirozeně patří k podnikům a část analytických dotazů se vztahuje právě k nim. [3][20]

Indexace v projektu

Vedle shardovacích klíčů jsou v projektu použity také sekundární indexy, které podporují nejčastější dotazovací vzory nad kolekcemi businesses, users a reviews. Indexy jsou vytvářeny

automaticky pomocí inicializační služby cluster-indexes, která se spouští po importu a normalizaci dat.

Základní indexace v projektu podporuje zejména filtrování podniků podle města, státu, hodnocení a počtu recenzí, práci s kategoriemi podniků, vyhledávání uživatelů podle aktivity, data registrace a počtu fanoušků, vyhledávání recenzí podle business_id, user_id, data, hodnocení a užitečnosti a základní textové vyhledávání v recenzích.

Příklady indexů vytvářených v rámci projektu jsou:

Kolekce businesses:

- idx_city_state_stars nad { city: 1, state: 1, stars: -1 },
- idx_categories nad { categories: 1 },
- idx_business_review_count nad { review_count: -1 }.

Kolekce users:

- idx_user_activity nad { review_count: -1, average_stars: -1 },
- idx_yelping_since nad { yelping_since: 1 },
- idx_user_fans nad { fans: -1 }.

Kolekce reviews:

- idx_business_date nad { business_id: 1, date: -1 },
- idx_user_date nad { user_id: 1, date: -1 },
- idx_review_rank nad { stars: -1, useful: -1 },
- txt_business_text nad { business_id: 1, text: "text" }.

Tyto indexy zrychlují typické dotazy používané v projektu, například filtrování podniků podle lokality, řazení podle hodnocení, vyhledávání recenzí konkrétního podniku nebo analýzu aktivity uživatelů. [13][14]

V kapitole **Dotazy** jsou navíc uvedeny demonstrační indexové příkazy. Ty ukazují, jak lze indexy vytvořit přímo v mongosh a následně ověřit jejich využití pomocí explain("executionStats"). V demonstračních dotazech jsou použity například compound indexy idx_reviews_business_date, idx_users_since_review_count a idx_businesses_city_state_open_stars.

Tyto demonstrační indexy nejsou v rozporu se základní indexací projektu. Slouží k tomu, aby bylo v sadě 30 dotazů ukázáno vytvoření indexu, praktický dotaz vracející data a ověření plánu vykonání dotazu pomocí explain("executionStats").

Zvolená indexace je pro potřeby semestrálního projektu dostačující. U produkčního nasazení nebo výrazně většího objemu dat by bylo vhodné indexy dále ladit podle skutečných přístupových vzorů, výstupů explain() a reálného zatížení databáze.

1.2.5 Replikace

Každý ze tří shardů je realizován jako tříčlenný replica set. Konkrétně: [4][11][17]

- shard01: shard1a, shard1b, shard1c,
- shard02: shard2a, shard2b, shard2c,
- shard03: shard3a, shard3b, shard3c.

Stejným principem je řešena i replikační sada config serverů:

- cfgReplSet: cfg1, cfg2, cfg3.

Replikační faktor 3 je pro projekt vhodný z několika důvodů: [4][11][17]

- umožňuje tolerovat výpadek jednoho uzlu bez ztráty majority,

- umožňuje automatickou volbu nového primárního uzlu,
- splňuje požadavek zadání na minimálně tři replikace,
- zároveň není zbytečně náročný na zdroje jako vyšší replikační faktory.

Jde o rozumný kompromis mezi odolností řešení a nároky na lokální provoz.

1.2.6 Perzistence dat

MongoDB v tomto řešení ukládá data perzistentně na disk prostřednictvím pojmenovaných Docker volumes. Každý databázový uzel má vlastní volume, takže data nezaniknou při běžném restartu kontejneru. [10][11]

Na úrovni storage engineu využívá MongoDB WiredTiger, který kombinuje: [10][11]

- primární paměti (RAM) – pro cache často používaných stránek a urychlení čtení,
- sekundární paměti (disk) – pro trvalé uložení dokumentů a žurnálu.

V konfiguračních souborech je navíc nastavena velikost cache WiredTiger na 0,5 GB pro config servery i shard servery, aby bylo chování uzlů v kontejnerovém prostředí předvídatelnější.

Načítání a ukládání dat v projektu probíhá takto:

1. Datové soubory jsou vloženy do složky Data/ na hostiteli.
2. Služba yelp-import přimountuje tuto složku jako read-only a pomocí mongoimport vloží JSON data do kolekcí.
3. Služba yelp-normalize následně vybraná pole převede do vhodnějších typů.
4. Zápisy běží přes mongos, který je směruje na primární uzly příslušných shardů.
5. Změny jsou replikovány na sekundární uzly v replikační sadě.

Projekt zatím neřeší samostatnou strategii zálohování a obnovy. Pro demonstrační nasazení je však perzistence přes volumes a replikační sady dostačující. [10][11]

1.2.7 Distribuce dat

Distribuce dat je v tomto řešení výsledkem kombinace shardingu a replikace. [2][4]

Tok požadavků při zápisu lze shrnout takto:

1. klient se připojí na mongos na portu 27017,
2. mongos podle metadat z config serverů zjistí, který shard je pro daný dokument relevantní,
3. zápis je proveden na primární uzel daného shardu,
4. změna je následně replikována na sekundární členy replikační sady.

Tok čtení je obdobný:

1. klient odešle dotaz na mongos,
2. mongos dotaz rozloží podle shard key nebo provede scatter/gather, pokud shard key není v dotazu přítomen,
3. výsledky z příslušných shardů sloučí a vrátí klientovi.

Klientské i administrační skripty se připojují přes mongos bez explicitní změny read preference, takže čtení běží standardně přes primární uzly. To zajišťuje předvídatelné chování dotazů.

Rozdělení dokumentů mezi shardy zajišťuje MongoDB balancer po zavedení shardingu nad kolekcemi. Díky hashed shard keys lze u kolekcí businesses a users očekávat poměrně rovnoměrné rozložení dat; u reviews tento přístup zároveň omezuje riziko hotspotů. [2][3][20]

```
[direct: mongos] test> use yelp
switched to db yelp
[direct: mongos] yelp> db.businesses.getShardDistribution()
Shard shard01 at shard01/shard1a:27018,shard1b:27018,shard1c:27018
{
  data: '41.04MiB',
  docs: 50302,
  chunks: 1,
  'estimated data per chunk': '41.04MiB',
  'estimated docs per chunk': 50302
}
---
Shard shard03 at shard03/shard3a:27018,shard3b:27018,shard3c:27018
{
  data: '40.86MiB',
  docs: 50044,
  chunks: 1,
  'estimated data per chunk': '40.86MiB',
  'estimated docs per chunk': 50044
}
---
Shard shard02 at shard02/shard2a:27018,shard2b:27018,shard2c:27018
{
  data: '40.84MiB',
  docs: 50000,
  chunks: 1,
  'estimated data per chunk': '40.84MiB',
  'estimated docs per chunk': 50000
}
---
Totals
{
  data: '122.74MiB',
  docs: 150346,
  chunks: 3,
  'Shard shard01': [
    '33.43 % data',
    '33.45 % docs in cluster',
    '855B avg obj size on shard'
  ],
  'Shard shard03': [
    '33.28 % data',
    '33.28 % docs in cluster',
    '856B avg obj size on shard'
  ],
  'Shard shard02': [
    '33.27 % data',
    '33.25 % docs in cluster',
    '856B avg obj size on shard'
  ]
}
```

Obrázek 3 - Shardované MongoDB - businesses

```
[direct: mongos] yelp> db.users.getShardDistribution()
Shard shard02 at shard02/shard2a:27018,shard2b:27018,shard2c:27018
{
  data: '138.41MiB',
  docs: 32910,
  chunks: 1,
  'estimated data per chunk': '138.41MiB',
  'estimated docs per chunk': 32910
}
---
Shard shard03 at shard03/shard3a:27018,shard3b:27018,shard3c:27018
{
  data: '144.07MiB',
  docs: 33803,
  chunks: 1,
  'estimated data per chunk': '144.07MiB',
  'estimated docs per chunk': 33803
}
---
Shard shard01 at shard01/shard1a:27018,shard1b:27018,shard1c:27018
{
  data: '140.78MiB',
  docs: 33287,
  chunks: 1,
  'estimated data per chunk': '140.78MiB',
  'estimated docs per chunk': 33287
}
---
Totals
{
  data: '423.28MiB',
  docs: 100000,
  chunks: 3,
  'Shard shard02': [
    '32.7 % data',
    '32.91 % docs in cluster',
    '4KiB avg obj size on shard'
  ],
  'Shard shard03': [
    '34.03 % data',
    '33.8 % docs in cluster',
    '4KiB avg obj size on shard'
  ],
  'Shard shard01': [
    '33.26 % data',
    '33.28 % docs in cluster',
    '4KiB avg obj size on shard'
  ]
}
```

Obrázek 4- Shardované MongoDB - users

```
[direct: mongos] yelp> db.reviews.getShardDistribution()
Shard shard02 at shard02/shard2a:27018,shard2b:27018,shard2c:27018
{
  data: '121.2MiB',
  docs: 165523,
  chunks: 1,
  'estimated data per chunk': '121.2MiB',
  'estimated docs per chunk': 165523
}
---
Shard shard03 at shard03/shard3a:27018,shard3b:27018,shard3c:27018
{
  data: '122.46MiB',
  docs: 168382,
  chunks: 1,
  'estimated data per chunk': '122.46MiB',
  'estimated docs per chunk': 168382
}
---
Shard shard01 at shard01/shard1a:27018,shard1b:27018,shard1c:27018
{
  data: '122.02MiB',
  docs: 166095,
  chunks: 1,
  'estimated data per chunk': '122.02MiB',
  'estimated docs per chunk': 166095
}
---
Totals
{
  data: '365.68MiB',
  docs: 500000,
  chunks: 3,
  'Shard shard02': [
    '33.14 % data',
    '33.1 % docs in cluster',
    '767B avg obj size on shard'
  ],
  'Shard shard03': [
    '33.48 % data',
    '33.67 % docs in cluster',
    '762B avg obj size on shard'
  ],
  'Shard shard01': [
    '33.36 % data',
    '33.21 % docs in cluster',
    '770B avg obj size on shard'
  ]
}
```

Obrázek 2 - Shardované MongoDB - reviews

1.2.8 Zabezpečení

Zabezpečení clusteru je řešeno na dvou úrovních:

1. interní autentizace mezi uzly pomocí keyfile,
2. uživatelská autentizace a autorizace.

Klíčovou částí je dynamicky generovaný keyfile. Služba keyfile-init při prvním spuštění vytvoří náhodný obsah keyfile, uloží jej do sdíleného volume mongo-secrets a databázové uzly jej následně používají pro interní autentizaci v replikačních sadách i shardovaném clusteru. [15][16]

Na uživatelské úrovni jsou automaticky vytvořeny dva účty:

- administrátorský účet v databázi admin (clusterAdmin),
- aplikační účet v databázi yelp (yelp_app).

Aplikační uživatel má role:

- readWrite nad databází yelp,
- dbAdmin nad databází yelp,
- read nad databází config.

Tato oprávnění jsou pro projekt dostačující, protože umožňují práci s daty i omezenou inspekci metadat, ale neudělují plná administrátorská práva nad celým clusterem. [15][16]

Řešení zatím nepoužívá TLS a je určeno pro lokální nebo laboratorní prostředí. Pro produkční nasazení by bylo vhodné zabezpečení rozšířit o šifrování komunikace, bezpečnější práci s tajemstvími a oddělení administrační a aplikační vrstvy. [18]

2. FUNKČNÍ ŘEŠENÍ

2.1 Struktura

Projekt je rozdělen do čtyř hlavních částí:

- Data/ – vstupní datasety a Python notebook pro analýzu,
- Dotazy/ – soubor se sadou MongoDB dotazů,
- Dokumentace/ – text dokumentace a pomocné poznámky,
- Funkcni_reseni/ – vlastní spustitelné řešení clusteru.

Detailnější popis:

Data/ slouží jako vstupní místo pro tři JSON datasety z Yelp Open Dataset a současně obsahuje notebook pro základní analýzu dat. [9]

Dotazy/ obsahuje soubor s MongoDB dotazy

Dokumentace/ obsahuje samotný text práce a pomocné soubory.

Funcni_reseni/ obsahuje vše potřebné pro automatizované spuštění clusteru:

- docker-compose.yml
- .env.example
- složku config/ s konfiguračními soubory MongoDB
- složku scripts/ s bootstrap a administračními skripty
- složku docker/mongo-tools/ s Dockerfile pro importní image

2.1.1 docker-compose.yml

Soubor docker-compose.yml je centrálním konfiguračním bodem celého řešení. Definuje služby, volumes, síť i pořadí inicializace. [8]

a) Služby generující a sdílející bezpečnostní tajemství

- keyfile-init je jednorázová služba založená na image python:3.12-slim. Vytváří ve sdíleném volume mongo-secrets keyfile pro interní autentizaci MongoDB uzlů. [16]

b) Konfigurační servery

- cfg1, cfg2 a cfg3 jsou config servery na image mongo:8.2. Používají konfiguraci mongod-csrs.conf, běží na portu 27019 a mají vlastní persistentní volumes. [4][8]
- cfg-init je jednorázová služba běžící v network namespace cfg1, která inicializuje replikační sadu config serverů cfgReplSet.

c) Datové uzly shardů

- shard1a, shard1b, shard1c
- shard2a, shard2b, shard2c
- shard3a, shard3b, shard3c

Datové uzly shardů používají konfiguraci mongod-shard.conf, vlastní persistentní volume a parametr --replSet, kterým je určen konkrétní shard.

- shard1-init, shard2-init a shard3-init jsou jednorázové služby inicializující replikační sady jednotlivých shardů.

d) Router a bootstrap clusteru

- mongos je router clusteru založený na image mongo:8.2. Používá konfiguraci mongos.conf, připojuje se na replikační sadu config serverů a publikuje port 27017 do hostitelského systému. [2][8]
- cluster-bootstrap nejprve přes localhost exception vytvoří administrátorského uživatele na mongos, následně se autentizuje a přidá do clusteru všechny tři shardy.

e) Databázové schéma a uživatelé

- cluster-schema po úspěšném bootstrapu vytvoří kolekce businesses, users a reviews, aplikuje validační schémata, zapne sharding databáze yelp, sharduje kolekce a vytvoří aplikačního uživatele yelp_app. [5]

f) Import a transformace dat

- yelp-import je buildovaná služba z vlastního Dockerfile. Mountuje složku ../Data jako read-only a pomocí mongoimport nahrává tři JSON soubory do kolekcí businesses, users a reviews. [9]
- yelp-normalize provádí post-importní normalizaci vybraných polí:
– businesses.categories: string → pole, – users.yelping_since: string → date, – users.friends: string / None → pole, – users.elite: string / None → pole, – reviews.date: string → date.

g) Indexace a GUI

- cluster-indexes vytváří sekundární indexy a textový index potřebné pro analytické dotazy.
- compass-web je webové GUI na image haohanyang/compass-web:0.3.1 publikované na portu 8081. Umožňuje pohodlnou kontrolu kolekcí a připojení přes MongoDB URI. [7]

h) Volumes, síť a závislosti

- Projekt využívá pojmenované Docker volumes pro config servery, shard uzly i sdílený secret volume.
- Všechny služby jsou součástí sítě mongo-net typu bridge.
- Pořadí inicializace je řízeno přes depends_on a condition: service_completed_successfully, takže bootstrap probíhá ve správném sledu bez ručního zásahu.

2.2 Instalace

Řešení je navrženo tak, aby první spuštění bylo maximálně automatizované. [8]

Požadavky

- Docker
- Docker Compose plugin
- dostatek volné RAM a místa na disku pro běžící cluster a import datasetů

Postup spuštění

1. Přejít do složky Funkcni_reseni/.
2. Vytvořit lokální .env soubor:

```
cp .env.example .env
```

3. Zajistit spustitelnost shell skriptů:

```
chmod +x scripts/*.sh
```

4. Spustit celé řešení:

```
docker compose up -d --build
```

5. Pro sledování jednorázových inicializačních kontejnerů použít:

```
docker compose logs -f keyfile-init cfg-init shard1-init shard2-init shard3-init
```

cluster-bootstrap cluster-schema yelp-import yelp-normalize cluster-indexes
6. Po dokončení ověřit stav služeb:

docker compose ps

7. Připojit se do mongos a ověřit počty dokumentů a stav shardingu:

```
docker exec -it mongos mongosh -u clusterAdmin -p 'ChangeMeRoot123!' \  
--authenticationDatabase admin
```

V shellu:

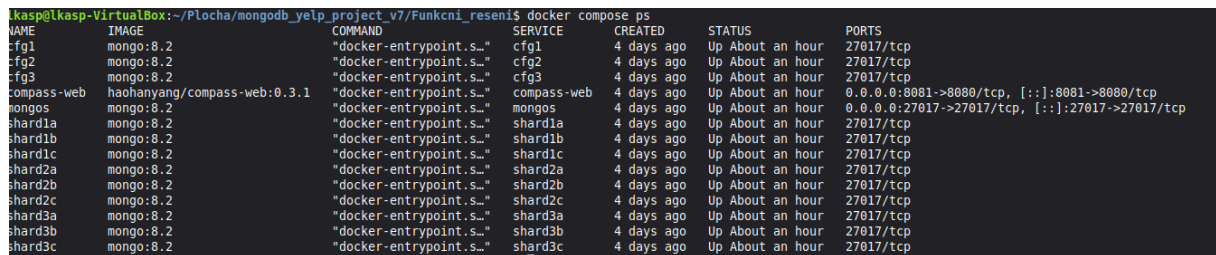
use yelp

db.businesses.countDocuments()

db.users.countDocuments()

db.reviews.countDocuments()

sh.status()



NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
cfg1	mongo:8.2	"docker-entrypoint.s..."	cfg1	4 days ago	Up About an hour	27017/tcp
cfg2	mongo:8.2	"docker-entrypoint.s..."	cfg2	4 days ago	Up About an hour	27017/tcp
cfg3	mongo:8.2	"docker-entrypoint.s..."	cfg3	4 days ago	Up About an hour	27017/tcp
compass-web	haohanyang/compass-web:0.3.1	"docker-entrypoint.s..."	compass-web	4 days ago	Up About an hour	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mongos	mongo:8.2	"docker-entrypoint.s..."	mongos	4 days ago	Up About an hour	0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
shard1a	mongo:8.2	"docker-entrypoint.s..."	shard1a	4 days ago	Up About an hour	27017/tcp
shard1b	mongo:8.2	"docker-entrypoint.s..."	shard1b	4 days ago	Up About an hour	27017/tcp
shard1c	mongo:8.2	"docker-entrypoint.s..."	shard1c	4 days ago	Up About an hour	27017/tcp
shard2a	mongo:8.2	"docker-entrypoint.s..."	shard2a	4 days ago	Up About an hour	27017/tcp
shard2b	mongo:8.2	"docker-entrypoint.s..."	shard2b	4 days ago	Up About an hour	27017/tcp
shard2c	mongo:8.2	"docker-entrypoint.s..."	shard2c	4 days ago	Up About an hour	27017/tcp
shard3a	mongo:8.2	"docker-entrypoint.s..."	shard3a	4 days ago	Up About an hour	27017/tcp
shard3b	mongo:8.2	"docker-entrypoint.s..."	shard3b	4 days ago	Up About an hour	27017/tcp
shard3c	mongo:8.2	"docker-entrypoint.s..."	shard3c	4 days ago	Up About an hour	27017/tcp

Obrázek 5 - Stav Docker služeb po úspěšném compose

Poznámka k opětovnému spuštění

Pro opětovné spuštění a zastavení lze využít pomocné skripty start_runtime.sh a stop_runtime.sh ve složce Funkcni_reseni.

```
chmod +x start-runtime.sh stop-runtime.sh
```

```
cd ~/Plocha/mongodb_yelp_project_v7/Funkcni_reseni
```

```
./start-runtime.sh
```

```
cd ~/Plocha/mongodb_yelp_project_v7/Funkcni_reseni
```

```
./stop-runtime.sh
```

3. PŘÍPADY UŽITÍ A PŘÍPADOVÉ STUDIE

3.1 Pro jaké účely je MongoDB vhodná

MongoDB je dokumentově orientovaná NoSQL databáze vhodná zejména pro případy, kdy: [1][2][4][13]

- data mají polostrukturovaný nebo proměnlivý charakter,
- je potřeba ukládat JSON/BSON dokumenty s vnořenými strukturami,
- aplikace pracuje s velkým objemem dat a může využít horizontální škálování,
- je potřeba kombinovat transakčnější zápis s analytickými a agregačními dotazy,
- schéma se může v čase vyvíjet bez nákladných relačních migrací.

Typickými oblastmi použití jsou e-commerce, katalogové systémy, obsahové platformy, logovací a eventové systémy, uživatelská data webových aplikací nebo personalizační a doporučovací systémy. [1][2][4]

3.2 Proč byla zvolena MongoDB pro Yelp data

MongoDB byla pro tento projekt zvolena proto, že Yelp data mají přirozeně dokumentový charakter. Kolekce businesses obsahuje vnořené objekty a pole, users sociální a historická metadata a reviews kombinují strukturovaná pole s delším textem recenze. [1][2][3][9][13]

Pro projekt jsou důležité zejména tyto výhody:

- přirozené uložení JSON dokumentů bez nutnosti jejich rozkladu do mnoha relačních tabulek,
- podporu agregací nad velkým objemem recenzí,
- možnost shardovat největší kolekce,
- jednoduché propojení kolekcí přes business_id a user_id,
- podporu indexace nad textem i nad klasickými atributy.

Vzhledem k tomu, že projekt sleduje nejen ukládání dat, ale i analytické dotazy nad recenzemi, uživateli a podniky, je dokumentová databáze pro tento scénář vhodnou volbou. [1][2][13]

3.3 Proč nebyla zvolena jiná NoSQL databáze

Redis / Valkey

Redis ani Valkey nejsou pro tento projekt ideální volbou. Jde primárně o in-memory key-value databáze vhodné pro cache, session storage a jednoduché operace, nikoli pro objemné dokumenty s agregacemi a propojováním kolekcí. [1]

Apache Cassandra

Cassandra je silná v distribuovaném zápisu a vysoké dostupnosti, ale obvykle vyžaduje dotazy navržené dopředu podle konkrétních access patternů. Pro projekt s rozmanitými analytickými dotazy je flexibilita MongoDB praktičtější. [2][4]

Couchbase / jiná dokumentová úložiště

Alternativou by mohly být i jiné dokumentové databáze, například Couchbase. MongoDB však nabízí silný ekosystém, známý aggregation framework a dobře popsání vzory pro shardovaný cluster v kontejnerovém prostředí. [1][13]

3.4 Případové studie

3.4.1 The Weather Channel

- **Odvětví:** Média, digitální obsah, meteorologické služby

- **Použité produkty:** MongoDB
- **Use Case:** Personalizace obsahu, mobilní aplikace, real-time upozornění a vysoko-zátěžový provoz

Výchozí stav

The Weather Channel provozoval rozsáhlé digitální služby pro web i mobilní zařízení a potřeboval zvládat velmi vysoký objem požadavků. Původní relační backend zpomaloval vývoj i nasazování nových funkcí a komplikoval personalizaci služeb v reálném čase. [19]

Řešení s MongoDB

Firma modernizovala své digitální služby s využitím MongoDB. Flexibilní dokumentový model zjednodušil práci s daty napříč webem, mobilem i dalšími kanály a umožnil rychlejší rozvoj aplikací. [19]

Výsledky

Migrace na MongoDB vedla k výraznému zrychlení vývoje a zjednodušení architektury. Případ ukazuje vhodnost MongoDB pro aplikace s vysokou zátěží, personalizací a častými změnami datového modelu. [19]

3.4.2 Bosch Digital

- **Odvětví:** Průmysl, IoT, digitální inovace
- **Použité produkty:** MongoDB
- **Use Case:** Ukládání, správa a analýza dat v reálném čase, IoT a big data scénáře

Výchozí stav

Bosch Digital pracuje s rozsáhlými a různorodými daty vznikajícími v prostředí internetu věcí. Takový scénář je náročný tím, že kombinuje velký objem dat, proměnlivou strukturu záznamů a potřebu průběžné analýzy. [20]

Řešení s MongoDB

Bosch Digital využívá MongoDB jako databázový základ pro IoT a analytické scénáře. Dokumentový model usnadňuje mapování dat ze zařízení na dokumenty a dynamické schéma podporuje další rozvoj aplikací bez rigidních změn modelu. [20]

Výsledky

Případová studie ukazuje MongoDB jako platformu vhodnou pro heterogenní data, průběžnou analytiku a rychlé budování nových use casů. Pro projekt je inspirativní zejména práce s proměnlivou strukturou dokumentů a větším objemem záznamů. [20]

3.4.3 eBay

- **Odvětví:** E-commerce, digitální platformy
- **Použité produkty:** MongoDB
- **Use Case:** Mission-critical aplikace, multi-data-center architektura, vysoká dostupnost a škálování

Výchozí stav

eBay je globální digitální platforma s velmi vysokými nároky na výkon, dostupnost a škálování. V takovém prostředí není možné tolerovat výpadky ani architekturu, která by nešla pružně rozšiřovat. [21]

Řešení s MongoDB

MongoDB je u eBay využívána jako jeden z enterprise standardů pro zákaznický orientované

aplikace. Studie popisuje nasazení mission-critical aplikací v multi-data-center prostředí, a to jak formou rozsáhlých replica setů, tak shardovaných clusterů. [21]

Výsledky

Případ eBay ukazuje, že MongoDB je vhodná i pro produkční provoz ve velkém měřítku, kde je nutné řešit současně škálování i vysokou dostupnost. Pro projekt je důležitý především jako potvrzení architektonického smyslu shardů, config serverů, mongos a replica setů. [21]

4. VÝHODY A NEVÝHODY

4.1 Výhody MongoDB obecně

- flexibilní dokumentové schéma, [1][2][4][13]
- přirozená práce s JSON/BSON daty,
- silný aggregation framework,
- možnost horizontálního škálování pomocí shardingu,
- vysoká dostupnost díky replikačním sadám,
- možnost kombinace klasických, složených i textových indexů,
- dobrá podpora pro rychlý vývoj a iterativní práci s datovým modelem.

4.2 Nevýhody MongoDB obecně

- složitější provoz u shardovaného clusteru oproti jedné instanci, [3][13][18]
- nutnost pečlivého návrhu shard key a indexů,
- scatter/gather dotazy mohou být nákladné, pokud nevyužívají shard key,
- produkční zabezpečení a monitoring vyžadují další vrstvy konfigurace,
- pro silně relační model s množstvím transakcí může být vhodnější klasická relační databáze.

4.3 Výhody tohoto konkrétního řešení

- plně automatizovaný bootstrap clusteru v Docker Compose,
- dynamicky generovaný keyfile, tedy nefixní interní autentizace,
- validační schémata pro všechny hlavní kolekce,
- automatizovaný import a následná normalizace dat,
- shardování všech tří hlavních kolekcí,
- připravená sada sekundárních indexů pro analytické dotazy,
- možnost vizuální kontroly clusteru přes Compass Web.

4.4 Nevýhody tohoto konkrétního řešení

- pouze jeden mongos, tedy nižší redundance na routerové vrstvě,
- chybí TLS a produkční práce se secrets,
- chybí monitoring, backup a disaster recovery,
- řešení je pro malý demonstrační vzorek architektonicky poměrně robustní až naddimenzované,
- výkonová optimalizace ještě není doložena execution plans ani benchmarkingem,
- finální dokumentace zatím postrádá některé screenshoty a přesné metriky datasetu.

5. DALŠÍ SPECIFIKA

Řešení je postaveno podle doporučeného principu shardovaného MongoDB clusteru a nepoužívá nestandardní mechanismy. Přesto má několik vlastních specifik: [2][4][15]

1. Jednorázové helper kontejnery pro automatizaci životního cyklu. Celé zprovoznění clusteru je rozděleno do logických kroků, což zlepšuje přehlednost i laditelnost bootstrapu.
2. Dynamická tvorba keyfile. Keyfile není uložen jako pevný soubor v repozitáři, ale generuje se při spuštění.
3. Samostatná normalizační fáze po importu. Část transformací probíhá až nad uloženými dokumenty, takže importní pipeline zůstává jednoduchá a datové úpravy jsou dobře dohledatelné.
4. Vlastní importní image s MongoDB Database Tools. Import je oddělen do specializovaného kontejneru, čímž se databázové uzly drží čisté a importní logika zůstává izolovaná.

6. DATA

6.1 Použité datasety

Řešení pracuje se třemi vzájemně propojitelnými soubory z Yelp Open Dataset: [9]

- yelp_academic_dataset_business.json
- yelp_academic_dataset_user.json
- yelp_academic_dataset_review.json

Tyto datasety popisují jedno společné téma – uživatele Yelp, podniky a jejich recenze. Propojitelnost zajišťují identifikátory:

- business_id
- user_id

6.2 Typy dat a formát

Vstupní data jsou ve formátu JSON Lines, tedy každý řádek představuje samostatný JSON dokument. MongoDB tato data při importu ukládá jako BSON dokumenty. [1][9]

Databáze zde pracuje zejména s těmito typy dat:

- řetězce (string) – názvy podniků, identifikátory, texty recenzí,
- čísla (int, double) – hodnocení, počty recenzí, počty fanoušků,
- datumy (date) – datum recenze, datum registrace uživatele,
- pole (array) – kategorie, přátelé, elite roky,
- vnořené dokumenty (object) – atributy podniků, otevírací doba,
- logické hodnoty / číselné příznaky – např. is_open.

6.3 Proč nebyly zvoleny jiné datové struktury

Data jsou uložena do tří samostatných kolekcí. To je vhodnější než silné embedování všech souvisejících informací do jediné kolekce, protože: [1][2][3]

- recenze představují objemově největší a potenciálně neomezeně rostoucí entitu,
- vkládat všechny recenze jako embedded pole do businesses nebo users by vedlo k neúměrně velkým dokumentům,
- oddělené kolekce umožňují lepší shardování, indexaci a pružnější analytické dotazy,
- propojení přes \$lookup je pro daný typ analýz dostačující.

6.4 Rozsah dat

Kromě tří použitých souborů obsahuje Yelp Open Dataset i další soubory, například tip a checkin. Ty však nebyly využity, protože zadání vyžaduje práci minimálně se třemi propojitelnými datasety a zvolené trojice pokrývá jádro tématu projektu. [9]

Původní soubory byly poměrně rozsáhlé: business přibližně 116 MB, review asi 5 GB a user přibližně 3,3 GB. Z důvodu rychlejšího importu, snazšího testování a omezených zdrojů byly proto připraveny zmenšené verze review a user pomocí skriptu clean_db.py.

Soubor s podniky nebyl dále zmenšován, protože jeho velikost byla pro zpracování přijatelná. U review a user byla provedena redukce na přibližně 300–350 MB. Výsledný soubor reviews obsahuje 500 000 záznamů a soubor users 100 000 záznamů.

yelp_business_v1.json

Obrázek 6 - yelp_business_v1.json - Head

	missing_count	missing_pct
hours	23223	15.45
attributes	13744	9.14
categories	103	0.07
address	0	0.00
business_id	0	0.00
name	0	0.00
postal_code	0	0.00
state	0	0.00
city	0	0.00
latitude	0	0.00
review_count	0	0.00
stars	0	0.00
longitude	0	0.00
is_open	0	0.00

Obrázek 8 - yelp_business_v1.json - missing values

	review_id	user_id	business_id	stars	useful	funny	cool		text	date
0	KU_05udG6zpxOg-VcAEodg	mh_-eMZ6K5RLWhZyISBhwa	QXcfwVdWr-0vZS3_CbbESXw	3	0	0	0		If you decide to eat here, just be aware it is...	2018-07-07 22:09:11
1	BiTunyQ73aT9WbnpR9DZGw	OyoGae7OkPv6sYGZT5g7Q	7ATy7TlgB3kUjt4UM3lypQ	5	1	0	1		I've taken a lot of spin classes over the year...	2012-01-03 15:28:18
2	saUsU_uimxRiCVr67Z4Jig	8_gItfSiwiKvNbP2etR0A	YjUwPpL6HXG530lw-fb2A	3	0	0	0		Family diner. Had the buffet. Eclectic assortm...	2014-02-05 20:30:30
3	AqPFMeIE6RsU23_aueSxiA	_7bhU9Uuf5_-HHC_8gbuQ	cxX2SOes4o-43ZQk8kMRfA	5	1	0	1		Wow! Yummy, different, delicious. Our favo...	2015-01-04 00:01:03
4	Sx8TMOmwLwNBWer-0pcmoA	bcjbaE6dDog4jkYV91nclQ	e4Vwtrqf-wpJfwsygvdgX	4	1	0	1		Cute interior and owner (?) gave us tour of up...	2017-01-14 20:54:15

	missing_count	missing_pct
review_id	0	0.0
user_id	0	0.0
business_id	0	0.0
stars	0	0.0
useful	0	0.0
funny	0	0.0
cool	0	0.0
text	0	0.0
date	0	0.0

Obrázek 10 - yelp_review_v1.json - missing values

	user_id	name	review_count	yearling_since	useful	funny	cool	elite	friends	face	..._compliment_more	compliment_profile	compliment_note	compliment_like	compliment_note	compliment_plain	compliment_cool	compliment_funny	compliment_w
0	qV8C0d0y7G5mMnJmshL	Walker	583	2007-01-29 16-07-26	7217	1225	2791	2009,2010,2011,2012,2013,2014,2015,2016,2017,2018	NC5=0a5m0y7G5mMnJmshL pe4yZuChQm9G5mMnJmshL	2167	...	65	55	56	18	232	644	467	467
1	1j46g0y7G5mMnJmshL	Danield	433	2009-01-29 04-05-09	9301	13064	27284	2009,2010,2011,2012,2013,2014,2015,2016,2017,2018	1j46g0y7G5mMnJmshL pe4yZuChQm9G5mMnJmshL	1638	...	64	184	157	251	1847	7054	3131	3131
2	2w0w7G5mMnJmshL	Steph	665	2008-07-29 04-11-00	2086	1010	1883	2009,2010,2011,2012,2013	2w0w7G5mMnJmshL pe4yZuChQm9G5mMnJmshL	52	...	13	10	17	3	66	96	119	119
3	5J2w0y7G5mMnJmshL	Karen	234	2005-11-29 04-08-31	512	330	299	2009,2010,2011,2012,2013	5J2w0y7G5mMnJmshL pe4yZuChQm9G5mMnJmshL	28	...	1	6	2	12	16	26	26	26
4	M0M1y7G5mMnJmshL	NGO	79	2007-01-06 19-03-09	29	15	7	2009,2010,2011,2012,2013	M0M1y7G5mMnJmshL pe4yZuChQm9G5mMnJmshL	2	...	0	0	0	1	1	0	0	0

20

	review_count	useful	funny	cool	fans	average_stars	compliment_hot	compliment_more	compliment_profile	compliment_cute	compliment_list	compliment_note	compliment_plain	compliment_cool	compliment_funny	compliment_writer	compliment_photos
count	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000
mean	92.633500	217.217900	91.406500	131.098710	7.478960	3.853158	9.201240	1.526240	0.987990	0.676530	0.399460	7.350100	16.397630	14.679750	14.679750	5.997230	5.130220
std	235.343251	1635.731828	1058.049858	1401.437743	57.493123	0.681401	123.190998	21.728636	31.498351	12.554029	12.538627	69.760793	237.128593	172.564051	172.564051	70.320518	104.642959
min	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	10.000000	7.000000	1.000000	1.000000	0.000000	3.530000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
50%	27.000000	26.000000	5.000000	7.000000	1.000000	3.910000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
75%	80.000000	96.000000	24.000000	31.000000	4.000000	4.280000	1.000000	1.000000	0.000000	0.000000	0.000000	2.000000	3.000000	2.000000	2.000000	1.000000	0.000000
max	17473.000000	206296.000000	185823.000000	195814.000000	12497.000000	5.000000	12391.000000	4347.000000	7039.000000	1744.000000	2667.000000	8616.000000	28974.000000	13280.000000	13280.000000	9821.000000	14045.000000

Obrázek 13 - yelp_user_v1.json - stats

	missing_count	missing_pct
user_id	0	0.0
name	0	0.0
review_count	0	0.0
yelping_since	0	0.0
useful	0	0.0
funny	0	0.0
cool	0	0.0
elite	0	0.0
friends	0	0.0
fans	0	0.0
average_stars	0	0.0
compliment_hot	0	0.0
compliment_more	0	0.0
compliment_profile	0	0.0
compliment_cute	0	0.0
compliment_list	0	0.0
compliment_note	0	0.0
compliment_plain	0	0.0
compliment_cool	0	0.0
compliment_funny	0	0.0
compliment_writer	0	0.0
compliment_photos	0	0.0

Obrázek 14 - yelp_user_v1.json - missing values

6.5 Prázdné hodnoty

Prázdné hodnoty se ve vzorku výrazněji týkají pouze souboru yelp_business_v1.json, zejména polí hours, attributes a categories.

6.6 Úpravy dat

Při zpracování dat jsou v projektu prováděny tyto úpravy:

1. businesses.categories Pole kategorií je po importu převedeno ze stringu odděleného čárkami na skutečné pole.
2. users.yelping_since Datum registrace uživatele je převedeno ze stringu na date.
3. users.friends a users.elite Pole, která mohou být uložena jako string nebo jako text None, jsou převedena na normalizovaná pole.
4. reviews.date Datum recenze je převedeno ze stringu na date.

Tyto transformace umožňují jednodušší práci s agregacemi, operátorem \$unwind i datumovými filtry. [9]

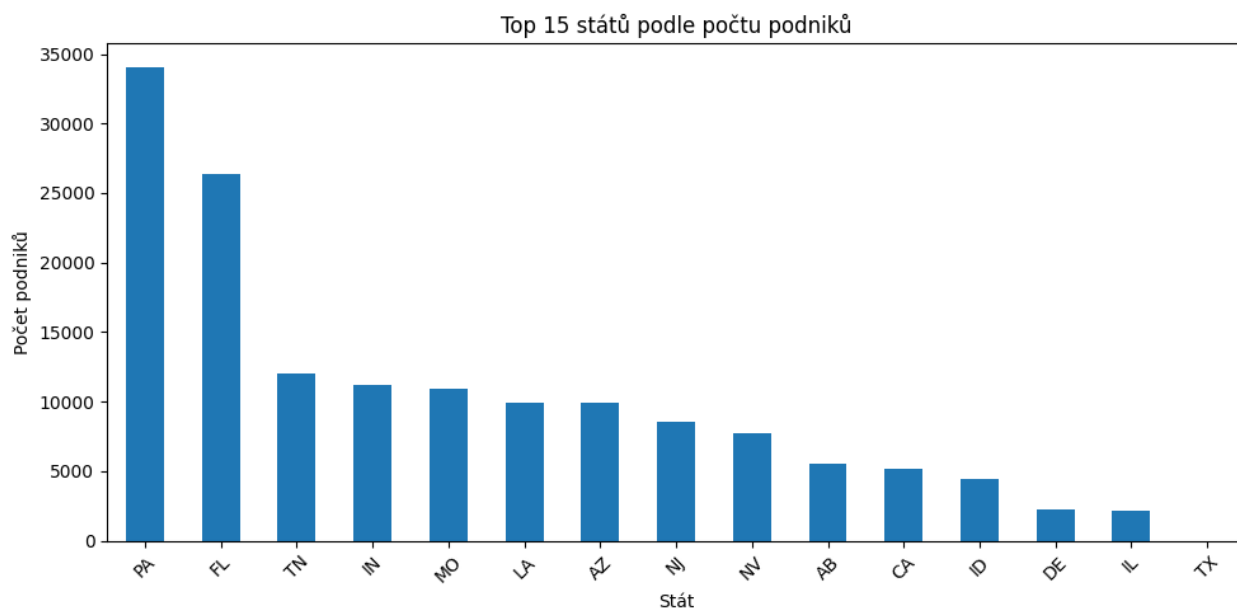
6.7 Zdroj dat

Zdroj dat je Yelp Open Dataset určený pro vzdělávací a výzkumné účely. [9]

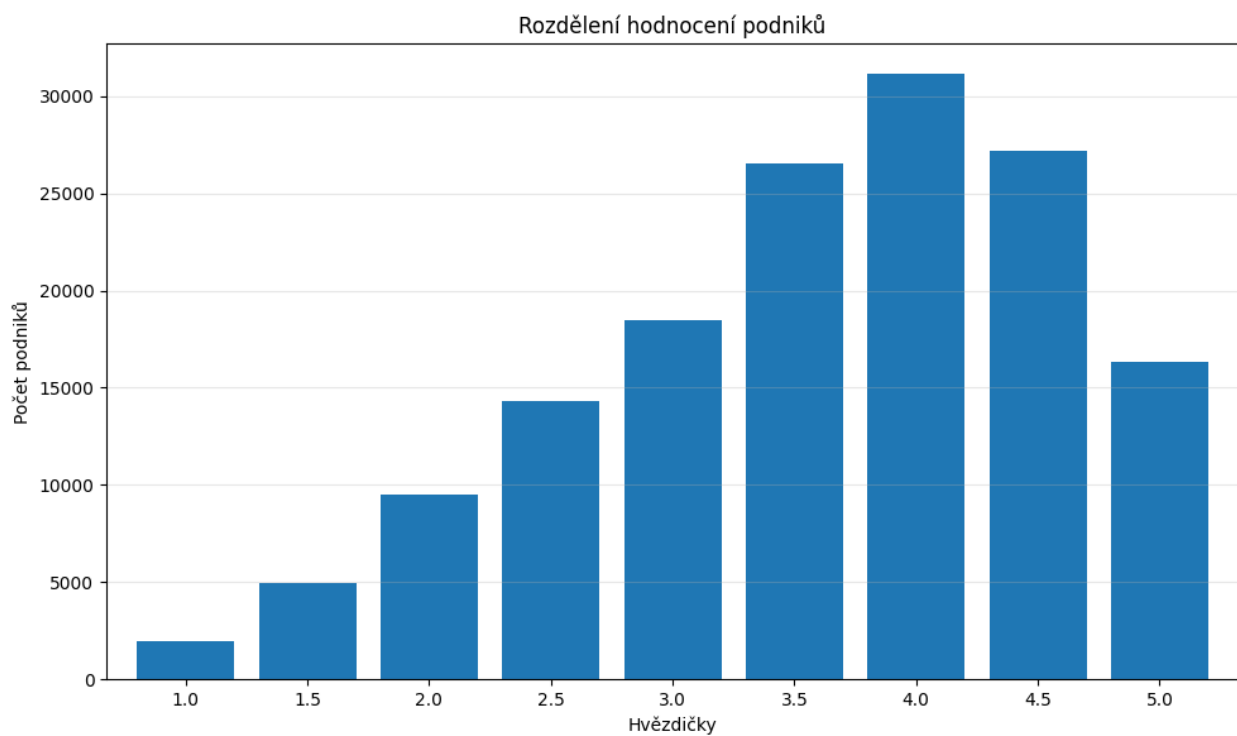
Oficiálním zdrojem dat je Yelp Open Dataset určený pro vzdělávací a výzkumné účely. [9]

6.8 Python analýza dat

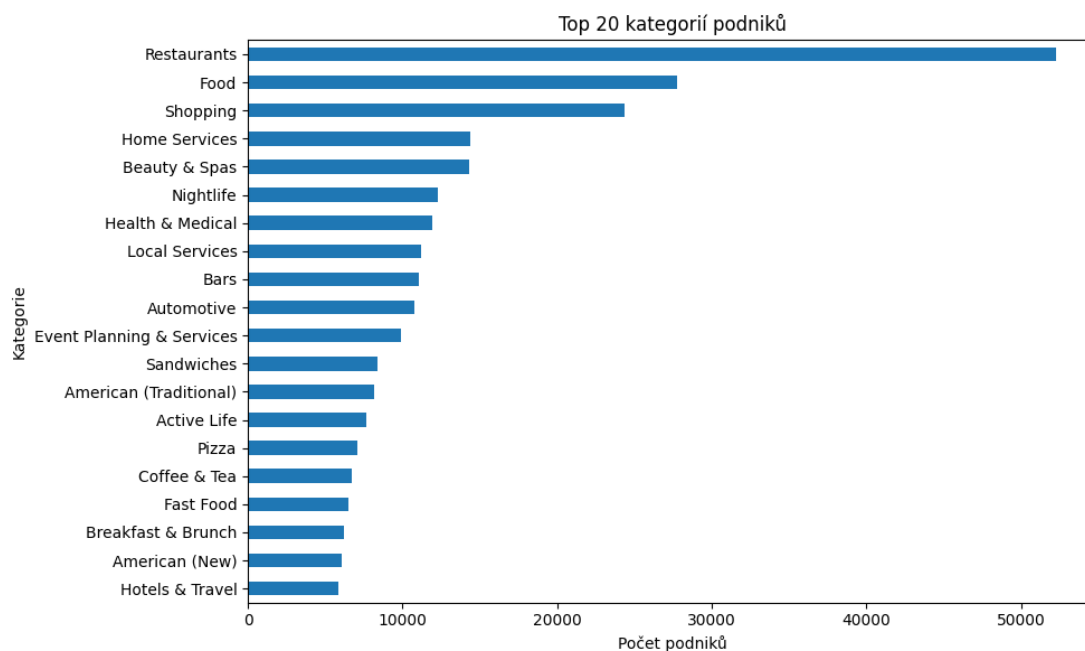
Součástí projektu je v příloze Jupyter notebook se základní explorativní analýzou s názvem `Explo – analyza.ipynb`



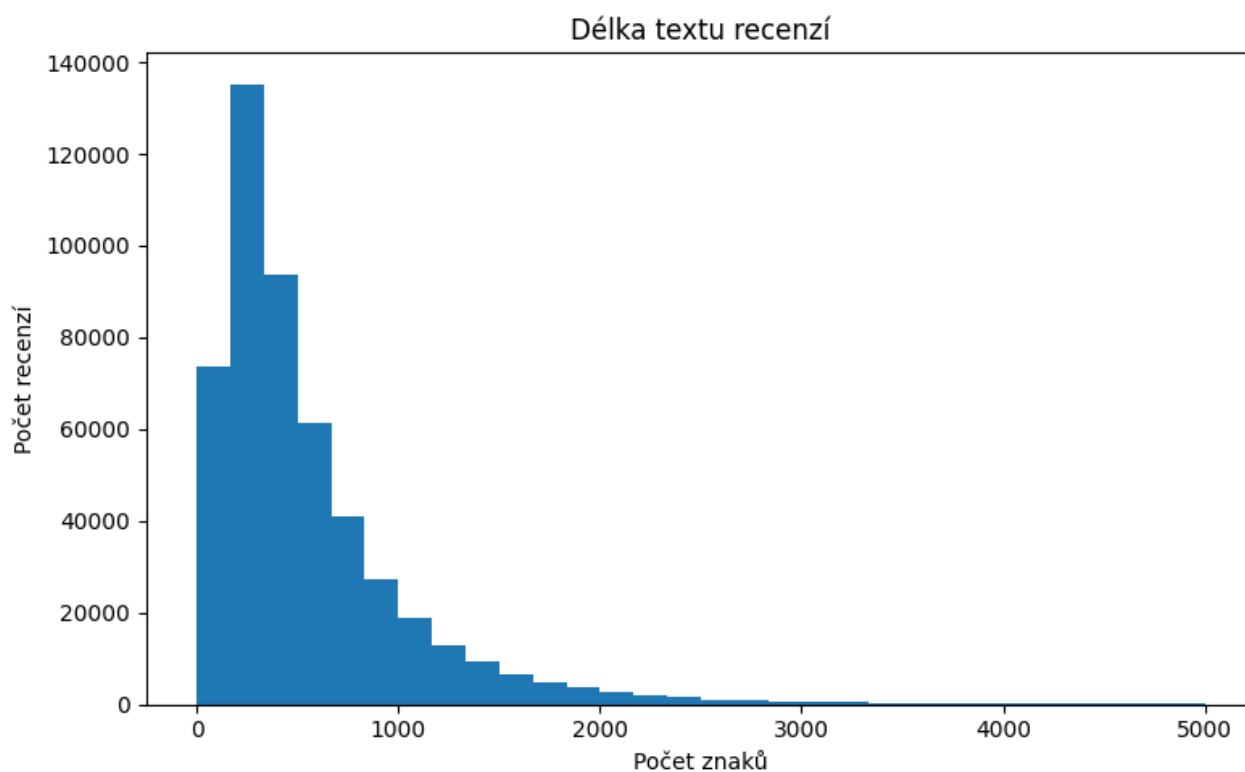
Obrázek 15 - Python analýza - Počet podniků



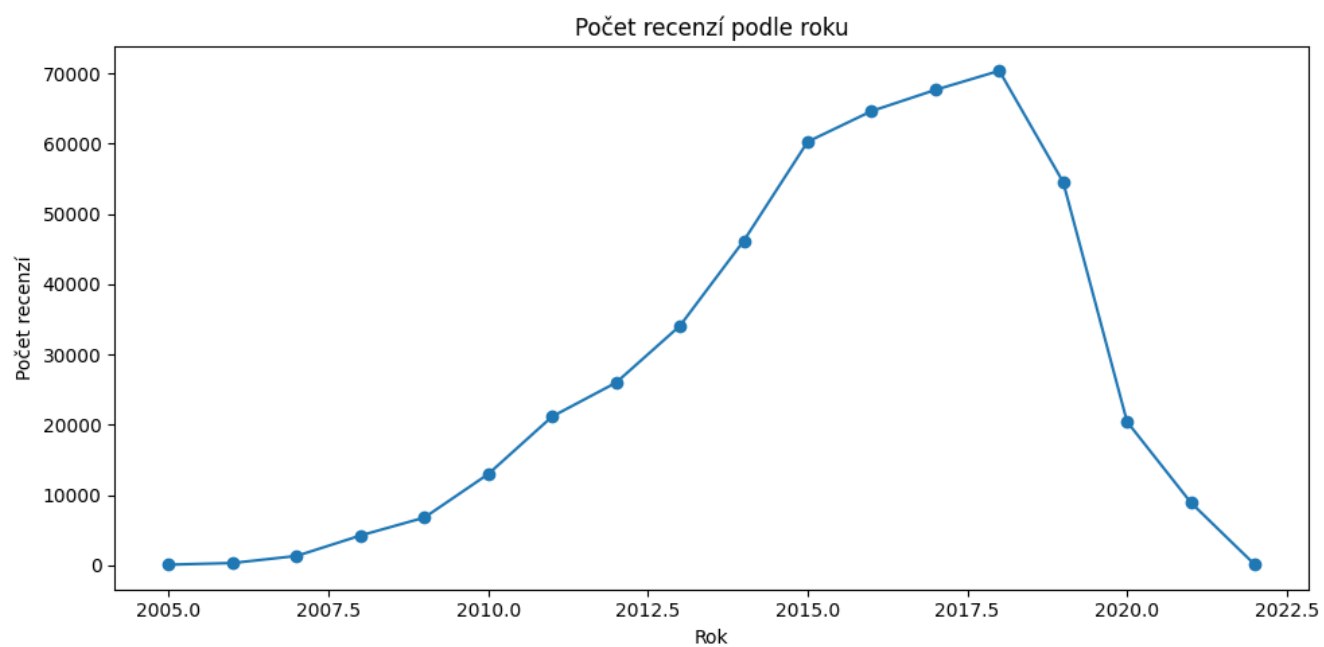
Obrázek 16 - Python analýza - Hodnocení podniků



Obrázek 17 - Python analýza - Top kat. podniků



Obrázek 18 - Python analýza - Délka textu recenzí



Obrázek 19 - Python analýza - Počet recenzí

7. DOTAZY

V rámci projektu byla připravena sada 30 netriviálních dotazů nad databází yelp. Dotazy jsou určeny pro spuštění v prostředí mongosh a pracují především se třemi hlavními kolekcemi businesses, reviews a users, které společně popisují podniky, uživatele a recenze z Yelp Open Datasetu. Dotazy jsou navrženy tak, aby na sebe částečně navazovaly a zároveň pokrývaly různé oblasti práce s MongoDB, včetně úprav dat, agregací, konfigurace clusteru, práce s vnořenými dokumenty a indexace. [1][13]

Součástí řešení je také validační schéma pro hlavní kolekce. Validací pravidla jsou nastavena v inicializačním skriptu configure-cluster-schema.sh, který je spuštěn službou cluster-schema v rámci Docker Compose. Tím je zajištěno, že databáze obsahuje JSON Schema validátory pro kolekce businesses, reviews a users.

Dotazy jsou rozděleny do pěti oblastí:

Práce s daty

Tato část ukazuje příkazy, které aktivně vytvářejí, upravují nebo mažou data. Patří sem například vložení záznamu pomocí insertOne, vytvoření pomocných analytických kolekcí pomocí agregačního operátoru \$merge, hromadná aktualizace přes updateMany a bezpečná ukázka mazání dat pomocí deleteMany. Destruktivní operace nejsou prováděny nad hlavními importovanými kolekcemi, ale nad pomocnými kolekcemi vytvořenými pro účely demonstrace.

Agregační funkce

Druhá část obsahuje analytické dotazy nad hlavními kolekcemi Yelp datasetu. Dotazy využívají zejména operátory aggregate, \$project, \$match, \$group, \$sort, \$limit, \$lookup a \$unwind. Pomocí těchto dotazů jsou zpracovány například nejlépe hodnocené kategorie podniků, aktivita uživatelů podle roku registrace, města s nejvyšším průměrným počtem recenzí nebo rozdíl mezi uloženým hodnocením podniku a průměrem vypočteným z recenzí.

Konfigurační dotazy

Třetí část je zaměřena na technickou konfiguraci MongoDB clusteru. Obsahuje výpis stavu shardovaného clusteru pomocí sh.status(), statistiky kolekcí, informace o shardovacích klíčích, indexech, validačních schématech a rozdělení dat mezi jednotlivé shardy. Tyto dotazy slouží jako důkaz, že databáze není provozována pouze jako jednoduchá lokální instance, ale jako shardovaný MongoDB cluster.

Práce s poli a vnořenými dokumenty

Čtvrtá oblast demonstruje práci s poli a embedded strukturami v dokumentech. V Yelp datasetu se jedná například o kategorie podniků, otevírací dobu, atributy podniků, seznam přátel uživatelů nebo roky, ve kterých byl uživatel označen jako elite. Dotazy využívají operátory jako \$unwind, \$size, \$ifNull, \$objectToArray, \$isArray, \$split a \$trim. Díky tomu jsou ošetřeny i rozdíly ve způsobu uložení některých polí, například jako pole nebo jako textový řetězec.

Indexy

Poslední část se věnuje vytváření a ověřování indexů. Jsou zde vytvořeny compound indexy nad kolekcemi reviews, users a businesses. Následně jsou ukázány praktické dotazy, které vrací konkrétní data, a zároveň je pomocí explain("executionStats") doloženo, jak MongoDB dotaz vykonává. Indexy jsou navrženy pro běžné přístupové vzory, například vyhledávání recenzí podle business_id, filtrování uživatelů podle data registrace a počtu recenzí nebo hledání podniků podle lokality, otevřenosti a hodnocení.

U většiny agregačních dotazů je použito nastavení:

```
{ allowDiskUse: true }
```

Toto nastavení umožňuje MongoDB při rozsáhlejších agregacích využít dočasné diskové úložiště pro mezivýsledky. Je vhodné zejména u dotazů, které provádějí seskupování, řazení, spojování kolekcí nebo zpracování většího množství dokumentů. [13]

Mimo hlavní sadu 30 dotazů je v dokumentaci zařazena také samostatná ukázka simulace výpadku uzlu replica setu. Tato část slouží k ověření chování clusteru při zastavení sekundárního uzlu shardu a jeho následném opětovném připojení. Tím je pokryt požadavek na práci s clusterem, replikačním faktorem a shardingem.

ZÁVĚR

Práce představuje funkční návrh a implementaci shardovaného MongoDB clusteru nad třemi propojitelnými datasety Yelp Open Dataset. [2][4][5][9]

Silnou stránkou řešení je automatizovaný bootstrap clusteru, oddělení inicializačních kroků do samostatných helper služeb, použití validačních schémat, dynamicky generovaného keyfile, shardingu hlavních kolekcí a připravené sady indexů pro analytické dotazy. [2][4][5][15]

Zvolená architektura je pro semestrální projekt plně dostačující a umožňuje prakticky demonstrovat nejdůležitější principy distribuované dokumentové databáze: replikaci, shardování, směrování požadavků, práci s validačními pravidly i import a transformaci polostrukturovaných dat. [2][4][11]

ZDROJE

[21] MONGODB. eBay building mission-critical multi-data-center applications with MongoDB [online]. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/company/blog/technical/ebay-building-mission-critical-multi-data-center-applications-with-mongodb>

[20] MONGODB. Bosch customer case study [online]. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/solutions/customer-case-studies/bosch>

[19] MONGODB. The Weather Channel customer case study [online]. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/solutions/customer-case-studies/weather-channel>

[18] MONGODB. Configure MongoDB Instances for TLS/SSL on Self-Managed Deployments [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/tutorial/configure-ssl/>

[17] MONGODB. Replica Set Elections [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/core/replica-set-elections/>

[16] MONGODB. Deploy Self-Managed Sharded Cluster with Keyfile Authentication [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/tutorial/deploy-sharded-cluster-with-keyfile-access-control/>

[15] MONGODB. Self-Managed Internal/Membership Authentication [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/core/security-internal-authentication/>

[14] MONGODB. Text Index Restrictions on Self-Managed Deployments [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/core/indexes/index-types/index-text/text-index-restrictions/>

- [13] MONGODB. Indexes [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/indexes/>
- [12] MONGODB. Manage Indexes [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/tutorial/manage-indexes/>
- [11] MONGODB. FAQ: Self-Managed MongoDB Diagnostics [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/faq/diagnostics/>
- [10] MONGODB. WiredTiger Storage Engine [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/core/wiredtiger/>
- [9] YELP. Open Dataset | Yelp Data Licensing [online]. [cit. 2026-05-01]. Dostupné z: <https://business.yelp.com/data/resources/open-dataset/>
- [8] DOCKER DOCS. Docker Compose [online]. [cit. 2026-05-01]. Dostupné z: <https://docs.docker.com/compose/>
- [7] DOCKER HUB. haohanyang/compass-web – Docker Image [online]. [cit. 2026-05-01]. Dostupné z: <https://hub.docker.com/r/haohanyang/compass-web>
- [6] DOCKER HUB. mongo – Official Image [online]. [cit. 2026-05-01]. Dostupné z: https://hub.docker.com/_/mongo
- [5] MONGODB. Specify JSON Schema Validation [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/core/schema-validation/specify-json-schema/>
- [4] MONGODB. Replication [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/replication/>
- [3] MONGODB. Choose a Shard Key [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/core/sharding-choose-a-shard-key/>
- [2] MONGODB. Sharding [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/sharding/>
- [1] MONGODB. What is MongoDB? [online]. MongoDB Documentation. [cit. 2026-05-01]. Dostupné z: <https://www.mongodb.com/docs/manual/>

PŘÍLOHY DOKUMENTACE

Data

Složka Data obsahuje tři datasey ve formátu JSON a Python notebook pro základní analýzu dat.

Dotazy

Složka Dotazy obsahuje jeden soubor se sadou 30 dotazů v MongoDB shell syntaxi včetně popisů.

Funkční řešení

Složka Funkcni_reseni obsahuje docker-compose.yml, konfigurační soubory MongoDB, bootstrap a inicializační skripty, Dockerfile pro importní image a další podpůrné soubory.